MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA TECHNICAL UNIVERSITY OF MOLDOVA

Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatic

REPORT ON LABORATORY WORK No. 4

Block Ciphers. The DES Algorithm Course: Cryptography and Security

Done by: student, group FAF-243
Andrei Elciscev

Checked by:
Zaica Maia

Chișinău, 2026

Contents

1 Introduction **3**

2 Objectives **3**

3 Theoretical Background **3**
3.1 General Characteristics of DES 3
3.2 General Structure of DES 4
3.3 Initial Permutation 4
3.4 Feistel Network 5
3.5 DES Round Function 5

4 DES Key Schedule **6**
4.1 Initial Key Permutation PC-1 6
4.2 Left Circular Shifts 6
4.3 Round-Key Generation: PC-2 7

5 Expansion, XOR and S-Box Operations **7**
5.1 Expansion Permutation 7
5.2 Exclusive OR 7
5.3 S-Boxes 8
5.4 Permutation P 9

6 Final Permutation and Ciphertext Reconstruction **9**

7 Implementation in C# **10**
7.1 Development Environment and Structure 10
7.2 Bit Representation 10
7.3 Key Schedule Implementation 10
7.4 DES Round Operations 11
7.5 Intermediate Results 11

8 Implemented Laboratory Task **12**
8.1 Task 2.11: Reconstruction of the Ciphertext 12

9 Testing and Results **12**
9.1 Test: Final Ciphertext Reconstruction 12
9.2 Testing Summary 13

10 Implementation Notes and Limitations **13**

11 Conclusions **14**

1 Introduction

Block ciphers are symmetric cryptographic algorithms that encrypt data in fixed-size blocks using a secret key. Unlike stream ciphers, which process data as a sequence of individual bits or symbols, block ciphers transform an entire block through a sequence of mathematical operations. The Data Encryption Standard (DES) is a classical symmetric-key block cipher standardized in the 1970s. DES processes 64-bit data blocks and uses a 64-bit key representation, of which 56 bits participate in the effective key schedule. The remaining eight bits are conventionally used as parity bits. DES uses a Feistel network consisting of sixteen rounds. Each round combines a half-block of data with a round key using expansion, XOR, substitution, permutation, and another XOR operation. Although DES is no longer considered secure for protecting modern sensitive information because its effective key length is only 56 bits, it remains useful for studying block-cipher design and the internal structure of symmetric encryption algorithms. This laboratory work focuses on Task 2.11: reconstructing the final DES ciphertext from the two 32-bit halves of round sixteen. The theoretical background describes the DES operations leading to that final reconstruction.

2 Objectives

The objectives of this laboratory work are:

- To study the structure and operation of the DES algorithm.
- To understand the role of the initial and final permutations.
- To examine the DES key schedule and the generation of round keys.
- To understand the expansion, XOR, substitution, and permutation operations.
- To implement selected DES operations in C#.
- To display intermediate calculations and the tables used by the algorithm.
- To support user-provided and randomly generated input values.
- To verify the final ciphertext reconstruction required in Task 2.11.

3 Theoretical Background

3.1 General Characteristics of DES

DES is a symmetric-key block cipher with the following principal characteristics:

- Block size: 64 bits.
- Key representation: 64 bits.
- Effective key length: 56 bits.
- Number of encryption rounds: 16.
- Round-key length: 48 bits.
- Internal data halves: 32 bits each.

The 64-bit key representation includes eight parity bits. The first key permutation, PC-1, selects 56 bits and removes the parity positions from the key schedule. The algorithm processes the input block through an initial permutation, sixteen Feistel rounds, a swap of the final halves, and a final permutation.

3.2 General Structure of DES

The principal stages of DES encryption are:

1. Convert the input block into a 64-bit binary representation.
2. Apply the initial permutation IP.
3. Divide the result into two 32-bit halves, L_0 and R_0.
4. Generate sixteen 48-bit round keys from the original key.
5. Execute sixteen Feistel rounds.
6. Concatenate the final halves in the order $R_{16}\|L_{16}$.
7. Apply the final permutation IP^{-1}.

The result is a 64-bit ciphertext block.

3.3 Initial Permutation

The initial permutation rearranges the bits of the input block according to a fixed table. It does not change the values of individual bits; it changes only their positions. The initial permutation table contains 64 entries. Each entry specifies the position of a bit in the original block that is copied into the corresponding output position. After applying the initial permutation, the resulting block is divided into two halves:

$$IP(M) = L_0\|R_0$$

where M is the original 64-bit message block. Both halves contain 32 bits.

3.4 Feistel Network

DES uses a Feistel structure in which each round combines the previous left half with a function of the previous right half and a round key. For round i, the equations are:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

where:

- L_i is the left half after round i;
- R_i is the right half after round i;
- K_i is the 48-bit round key;
- f is the DES round function;
- $\oplus$ denotes bitwise exclusive OR.

The Feistel structure allows encryption and decryption to use the same round structure, with the round keys applied in opposite order.

3.5 DES Round Function

The round function f takes a 32-bit right half and a 48-bit round key. It produces a 32-bit result. Its operations are:

1. Expand the 32-bit right half to 48 bits using the expansion permutation E.
2. XOR the expanded result with the round key.
3. Divide the 48-bit result into eight 6-bit blocks.
4. Apply one S-box to each block, producing eight 4-bit outputs.
5. Concatenate the eight outputs into a 32-bit value.
6. Apply the permutation P.

The round function can be written as:

$$f(R_{i-1}, K_i) = P(S(E(R_{i-1}) \oplus K_i))$$

Here, S represents the combined operation of the eight S-boxes.

4 DES Key Schedule

4.1 Initial Key Permutation PC-1

The DES key schedule begins by applying the Permuted Choice 1 table, also known as PC-1, to the 64-bit key.

$$K^+ = PC1(K)$$

The result K^+ contains 56 bits. The original key is rearranged, and the eight parity-bit positions are omitted. The resulting 56-bit value is divided into two 28-bit halves:

$$C_0 = K^+[1 \dots 28]$$

$$D_0 = K^+[29 \dots 56]$$

The values C_0 and D_0 are the starting points for generating the round keys.

4.2 Left Circular Shifts

During each round, both halves are rotated to the left. The number of positions shifted depends on the round number. The shift schedule is shown in Table 1.

Table 1: DES key-schedule shift schedule

Round	Shift	Round	Shift
1	1	9	1
2	1	10	2
3	2	11	2
4	2	12	2
5	2	13	2
6	2	14	2
7	2	15	2
8	2	16	1

A left circular shift moves the leftmost bits to the right end rather than discarding them. For example, if a bit string is represented by C, a one-position circular shift is:

$$LS_1(C) = C[2 \dots 28] \| C[1]$$

After the required shift:

$$C_i = LS_{s_i}(C_{i-1})$$

$$D_i = LS_{s_i}(D_{i-1})$$

where s_i is the shift count for round i.

4.3 Round-Key Generation: PC-2

After shifting the two 28-bit halves, they are concatenated:

$$C_i \| D_i$$

The Permuted Choice 2 table, or PC-2, selects and rearranges 48 bits from this 56-bit value.

$$K_i = PC2(C_i \| D_i)$$

The resulting K_i is the round key used in round i. The key schedule produces sixteen round keys:

$$K_1, K_2, \ldots, K_{16}$$

Each round key contains 48 bits.

5 Expansion, XOR and S-Box Operations

5.1 Expansion Permutation

The expansion permutation E converts the 32-bit right half into a 48-bit value. Certain bits are repeated so that the output can be combined with a 48-bit round key.

$$E : \{0,1\}^{32} \to \{0,1\}^{48}$$

The expanded value is XORed with the round key:

$$B = E(R_{i-1}) \oplus K_i$$

The result is 48 bits long and is divided into eight 6-bit blocks:

$$B = B_1 \| B_2 \| \cdots \| B_8$$

Each block is processed by its corresponding S-box.

5.2 Exclusive OR

The XOR operation compares two bits and returns 1 when the bits differ and 0 when they are equal.

Table 2: XOR truth table

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

For bit strings of equal length, XOR is applied independently to each corresponding pair of bits. For example:

$$1011 \oplus 1100 = 0111$$

5.3 S-Boxes

DES contains eight substitution boxes, designated S_1 through S_8. Each S-box maps a 6-bit input to a 4-bit output. Each S-box is represented by a table containing four rows and sixteen columns. For a six-bit input:

$$B_j = b_1 b_2 b_3 b_4 b_5 b_6$$

the row is determined by the first and last bits:

$$r = (b_1 b_6)_2$$

The column is determined by the four middle bits:

$$c = (b_2 b_3 b_4 b_5)_2$$

The selected table entry is:

$$v = S_j[r, c]$$

The value v, which ranges from 0 to 15, is represented as a four-bit binary string. For example, consider the input:

$$B_j = 011011$$

The row is obtained from the first and last bits:

$$r = (01)_2 = 1$$

The column is obtained from the middle four bits:

$$c = (1101)_2 = 13$$

The output is the value stored in row 1, column 13 of the corresponding S-box, written as four binary bits. DES S-box tables use row and column indices starting from zero. After all eight S-boxes are applied, their outputs are concatenated:

$$S(B) = S_1(B_1) \| \cdots \| S_8(B_8)$$

The result contains 32 bits.

5.4 Permutation P

The permutation P rearranges the 32 bits produced by the S-boxes.

$$P : \{0,1\}^{32} \to \{0,1\}^{32}$$

The permuted output is combined with the previous left half to obtain the next right half:

$$R_i = L_{i-1} \oplus P(S(B))$$

The next left half is:

$$L_i = R_{i-1}$$

6 Final Permutation and Ciphertext Reconstruction

After sixteen rounds, DES has produced L_{16} and R_{16}. The final permutation is applied after the halves are concatenated in reverse order. The 64-bit preoutput block is:

$$W = R_{16} \| L_{16}$$

The ciphertext block is then calculated as:

$$C = IP^{-1}(W)$$

The final permutation table IP^{-1} is the inverse of the initial permutation. It rearranges the bits but does not change their individual values. The final 64-bit result can be represented as a 16-digit hexadecimal string. Each hexadecimal digit corresponds to four bits. Task 2.11 performs the concatenation, applies the final permutation, and converts the resulting bits to hexadecimal.

7 Implementation in C#

7.1 Development Environment and Structure

The program was implemented in C# using the .NET environment. It provides a console menu through which the user can select one of the DES calculation tasks. The implementation is organized into several classes:

- `BitUtils` – binary and hexadecimal conversion, ASCII conversion, bit permutations, circular shifts, XOR, grouping, and random hexadecimal input generation.
- `DesTables` – the fixed DES tables, including the initial and final permutations, expansion permutation, PC-1, PC-2, shift schedule, permutation P, and all eight S-boxes.
- `DesKeySchedule` – generation of K^+, the C_i and D_i halves, and the round keys.
- `DesRoundFunction` – initial permutation processing, expansion, XOR, S-box calculations, permutation P, and round-result calculations.
- `DesFinalBlock` – reconstruction of the final ciphertext from L_{16} and R_{16}.
- `DesLabConsole` – the interactive menu, input processing, and display of intermediate results.

7.2 Bit Representation

The program represents binary values as strings containing the characters `0` and `1`. This makes intermediate values easy to display and inspect. The `HexToBits` method converts each hexadecimal digit into four binary bits. The `BitsToHex` method performs the reverse conversion. The `AsciiToBits` method converts each input character into an eight-bit representation. It is used when the user enters an eight-character key or message. The `Permute` method applies a permutation table by copying the input bits in the order specified by the table. Since the DES tables use one-based bit positions, the implementation subtracts one when accessing a string index.

7.3 Key Schedule Implementation

The `FromKeyBits` method applies PC-1 to the 64-bit key and separates the result into C_0 and D_0. For each of the sixteen rounds, the method:

1. Retrieves the required circular-shift count.
2. Rotates C and D to the left.

3. Concatenates the shifted halves.
4. Applies PC-2 to obtain the round key.
5. Stores the round number, shift count, halves, key bits, and hexadecimal key.

The `GetCiDi` method calculates the two key halves for a specified round. The `GetRoundKey` method applies PC-2 to the halves of that round. The `GetAllRoundKeys` method generates the complete set of sixteen round keys.

7.4 DES Round Operations

The `ExpandRightHalf` method applies the expansion permutation to a 32-bit right half. The `XorWithKey` method combines the expanded value with a 48-bit round key. The `SplitIntoSixBitBlocks` method divides the result into eight blocks. The `ApplySBoxes` method calculates the row and column for each block, looks up the value in the appropriate S-box, converts the result into four bits, and concatenates all outputs. The `ApplyPermutationP` method rearranges the 32-bit S-box result. The `ComputeRightHalf` method then XORs that value with the previous left half.

7.5 Intermediate Results

The program displays intermediate values to make the algorithm traceable. Depending on the selected task, the output may include:

- The input key or message in binary.
- The result of PC-1, K^+.
- The C_i and D_i halves.
- The number of circular shifts for each round.
- The 48-bit round keys in binary and hexadecimal.
- The initial permutation result and the L_0, R_0, and L_1 halves.
- The expanded right half and its XOR with the round key.
- The eight six-bit S-box inputs, row and column indices, lookup values, and four-bit outputs.
- The result of permutation P and the next right half.
- The final preoutput block and ciphertext.

These values allow the user to follow the calculations instead of seeing only the final answer.

8 Implemented Laboratory Task

The report focuses on Task 2.11, which reconstructs the final DES ciphertext from the two 32-bit halves produced by round sixteen.

8.1 Task 2.11: Reconstruction of the Ciphertext

The user enters L_{16} and R_{16}, each containing 32 bits. The program concatenates the halves in the required order:

$$W = R_{16} \| L_{16}$$

The final permutation is applied:

$$C = IP^{-1}(W)$$

The program displays L_{16}, R_{16}, the preoutput block, the final ciphertext bits, and the ciphertext in hexadecimal.

9 Testing and Results

Testing focuses on reconstructing the DES ciphertext from the two final round halves and applying the inverse initial permutation.

9.1 Test: Final Ciphertext Reconstruction

The final test uses the two 32-bit halves from the sixteenth round. They are concatenated in the order $R_{16} \| L_{16}$, after which the final permutation produces the 64-bit ciphertext.

```
DES - Task 2.11: ciphertext from L16 and R16
Enter each half as 32 bits, 8 hexadecimal digits, or R for random.

L16: r
Generated L16 = 90408B09
R16: r
Generated R16 = 6590C426

Final permutation IP⁻¹ table:
40  8 48 16 56 24 64 32
39  7 47 15 55 23 63 31
38  6 46 14 54 22 62 30
37  5 45 13 53 21 61 29
36  4 44 12 52 20 60 28
35  3 43 11 51 19 59 27
34  2 42 10 50 18 58 26
33  1 41  9 49 17 57 25

Intermediate steps:
L16       = 1001 0000 0100 0000 1000 1011 0000 1001
R16       = 0110 0101 1001 0000 1100 0100 0010 0110
R16 || L16 = 0110 0101 1001 0000 1100 0100 0010 0110 1001 0000 0100 0000 1000 1011 0000 1001
IP⁻¹(R16 || L16) = 0100 1010 0000 1001 0100 0101 0000 1010 1001 0000 0100 0001 0110 0100 1001 1100

Ciphertext (hex) = 4A09450A9041649C
```

Figure 1: Reconstruction of the final DES ciphertext

9.2 Testing Summary

The test demonstrates Task 2.11: concatenating the final DES round halves in the order $R_{16}\|L_{16}$, applying the final permutation IP^{-1}, and representing the result as a 64-bit ciphertext and a 16-digit hexadecimal value. The output can be checked against an independently calculated final permutation.

10 Implementation Notes and Limitations

The program is designed to demonstrate selected DES operations rather than provide a complete encryption interface that accepts a message and automatically performs all sixteen rounds. The user can calculate individual components and inspect their intermediate results. These components collectively illustrate the internal operations of the DES algorithm. The program represents bits as strings, which makes the calculations easier to inspect but is less efficient than using integer and bitwise operations for production cryptographic software. The ASCII input option is intended for eight-character input using single-byte character values. For hexadecimal input, each digit represents four bits, so a complete 64-bit block requires sixteen hexadecimal digits. For random key and message input, the program must generate sixteen hexadecimal digits to represent a full 64-bit value. Random values for other tasks must contain the number of hexadecimal digits corresponding to the required bit length. DES is included for educational purposes only. Its effective key length of 56 bits makes it vulnerable to brute-force attacks, and it should not be used to protect modern sensitive data.

11 Conclusions

During this laboratory work, the structure and principal operations of the DES block cipher were studied, with the report focusing on final ciphertext reconstruction in C# (Task 2.11). The application demonstrates the generation of the effective key, the separation and rotation of key halves, the generation of round keys, the expansion and XOR operations, S-box substitution, permutation P, the Feistel-round calculation, and final ciphertext reconstruction. The implementation displays intermediate values so that each stage can be examined and compared with the corresponding DES tables. The console interface also allows users to enter data manually or request random input for supported operations. The laboratory work provided practical experience with binary and hexadecimal representations, bit permutations, circular shifts, XOR operations, substitution tables, and the Feistel structure. Although DES is no longer suitable for modern security requirements, understanding its operation provides a useful foundation for studying block ciphers and more modern symmetric encryption algorithms.